OptiCrop: Smart Agricultural Production Optimization Engine

Project Report

Student Name: PENUMUDI PAVULAMMA
Roll No: 25X41DAI17
Guide:SMARTBRIDGE
Department: CSE-AIML
College: SRK INSTITUE OF TECHNOLOGY
YENIKEPADU,VIJAYAWADA

Entity Relationship (ER) Diagram

Description

The ER diagram represents the core entities involved in the OptiCrop Smart Agricultural Production
Optimization Engine and illustrates how they interact. It organizes user information, soil data, crop
Recommendations, datasets, machine learning models, predictions, and reports.

Entities Involved

User, SoilData, Crop, Dataset, MLModel, Prediction, Report.

Primary Keys

User:user_id, SoilData:soil_id, Crop:crop_id, Dataset:dataset_id, MLModel:model_id,
Prediction:prediction_id, Report:report_id.

Relationships

User→SoilData (1:M); SoilData→Prediction (1:1); Crop→Prediction (1:M); MLModel→Prediction
(1:M); Dataset→MLModel (1:M); Prediction→Report (1:M).

Foreign Keys

SoilData.user_id→User; Prediction.soil_id→SoilData;
Prediction.crop_id→Crop;
Prediction.model_id→MLModel; Report.prediction_id→Prediction.

Cardinality

One user may submit many soil records. Each soil record generates one prediction. Crops and ML
models may appear in many predictions. Each prediction may produce multiple reports.

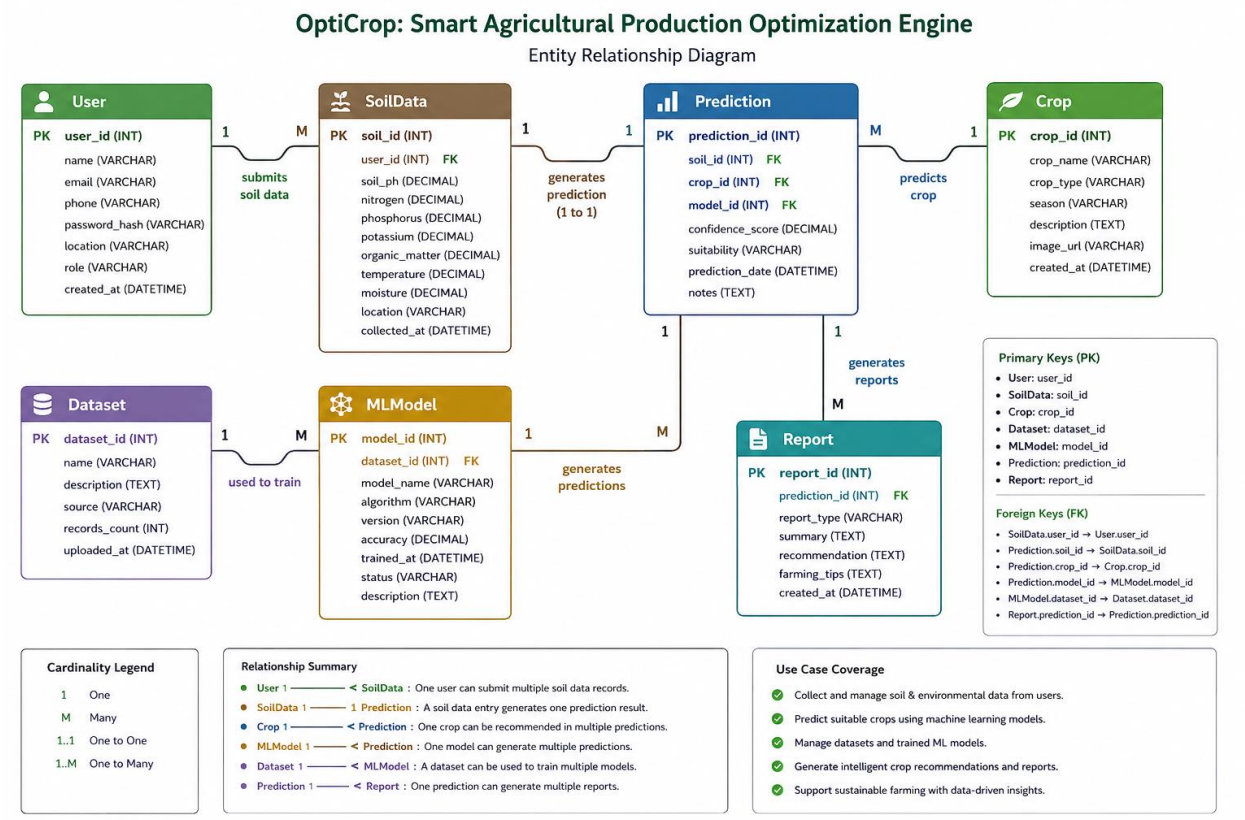
Normalization and Structure

Entities are separated to reduce redundancy, improve scalability, and simplify maintenance.

Use Case Coverage

Supports soil data collection, crop prediction, dataset and model management, report generation,
and intelligent farming decisions.

Sample ER Diagram (Conceptual)



Pre-requisites

The successful development, deployment, and execution of the **OptiCrop: Smart Agricultural Production Optimization Engine** require appropriate software, development tools, programming knowledge, and system resources. The following pre-requisites ensure smooth implementation and efficient performance of the application.

1. Software Requirements

- **Operating System:** Windows 10/11, Ubuntu 20.04+, or macOS
- **Database:** MySQL 8.0 or SQLite
- **Web Browser:** Google Chrome, Mozilla Firefox, or Microsoft Edge
- **Version Control:** Git (optional but recommended)

2. Python Libraries and Frameworks

The following libraries should be installed before running the project:

- Flask (Web Framework)
- Pandas (Data Processing)
- NumPy (Numerical Computation)
- Scikit-learn (Machine Learning Algorithms)
- Matplotlib (Data Visualization)
- Seaborn (Statistical Visualization)
- Joblib (Model Serialization)
- SQLAlchemy (Database Connectivity)
- Flask-SQLAlchemy (ORM)
- Flask-Login (User Authentication)
- Flask-WTF (Form Handling)

Install the required packages using:

```
pip install flask pandas numpy scikit-learn matplotlib seaborn joblib sqlalchemy flask-sq
```

3. Hardware Requirements

Minimum Configuration

- Processor: Intel Core i3 or AMD Equivalent
- RAM: 4 GB
- Storage: 10 GB Free Disk Space

- Internet Connection: Required for updates and deployment

Recommended Configuration

- Processor: Intel Core i5/i7 or AMD Ryzen 5+
- RAM: 8 GB or higher
- Storage: 20 GB SSD
- Stable Internet Connection

4. Dataset Requirements

The system requires an agricultural dataset containing soil and environmental parameterssuch as:

- Nitrogen (N)
- Phosphorus (P)
- Potassium (K)
- Soil pH
- Temperature
- Humidity
- Rainfall
- Crop Label

The dataset should be cleaned, validated, and formatted (CSV format recommended)before model training.

5. Machine Learning Requirements

The project assumes familiarity with:

- Data Preprocessing
- Feature Engineering

- Classification Algorithms
- Model Training and Testing
- Model Evaluation Metrics
- Model Serialization using Joblib

6. Database Requirements

The database should contain tables for:

- User
- Soil Data
- Crop
- Dataset
- ML Model
- Prediction
- Report

Primary and foreign key relationships should be configured according to the ER diagram.

7. Development Knowledge

Developers should have basic knowledge of:

- Python Programming
- Object-Oriented Programming (OOP)
- SQL and Database Design
- Flask Web Framework
- HTML, CSS, and JavaScript
- Machine Learning using Scikit-learn
- Data Visualization
- Git Version Control (optional)

8. System Configuration

Before executing the project:

- Install Python and all required libraries.
- Configure the database connection.
- Import the agricultural dataset.
- Train or load the machine learning model.
- Update application configuration files (database credentials, modelpaths, etc.).
- Run the Flask application.

Example:
Bash
```
python app.py
```
or
flask run

9. Resources

No additional external resources are mandatory for the development or execution of the project. All required software, libraries, and tools are open-source and can be installed locally. The agricultural dataset and trained machine learning model are sufficient to operate the **OptiCrop: Smart Agricultural Production Optimization Engine**.

Workflow

Description

The development workflow of the **OptiCrop: Smart Agricultural ProductionOptimization Engine** is organized into five major epics that cover the complete lifecycle ofthe project, from problem identification to application deployment. Each epic consists ofmultiple user stories that describe the activities required to successfully develop theintelligent crop recommendation system.

Epic 1: Problem Definition and Requirement Analysis

Story 1

Identify and define the agricultural problem that the crop recommendation system aims tosolve by understanding farmers' challenges in selecting suitable crops based on soil andenvironmental conditions.

Story 2

Gather and analyze functional and non-functional requirements to determine the projectobjectives, system scope, and expected outcomes.

Story 3

Conduct a comprehensive literature review to study existing crop recommendationsystems, machine learning techniques, and precision agriculture solutions.

Story 4

Analyze the business, environmental, and social impact of implementing an AI-driven croprecommendation system for sustainable farming and

improved agricultural productivity. **Epic 2: Data Collection and Exploratory Data Analysis**

Story 1

Collect and download agricultural datasets from reliable sources containing soil nutrients,climatic parameters, and crop labels.

Story 2

Import the required Python libraries for data manipulation, visualization, machine learning,and model development.

Story 3

Load the dataset and perform exploratory data analysis to understand feature distributions,data types, and target variables.

Story 4

Perform univariate analysis to examine the distribution of individual agricultural featuressuch as nitrogen, phosphorus, potassium, temperature, humidity, pH, and rainfall.

Story 5

Conduct bivariate analysis to identify relationships between soil parameters and cropsuitability.

Story 6

Perform multivariate analysis to discover hidden patterns and interactions among multipleagricultural variables that influence crop recommendations.

Epic 3: Data Pre-processing

Story 1

Inspect the dataset for missing values and apply suitable techniques to handle incompleteor inconsistent data.

Story 2

Detect and treat outliers to improve data quality and enhance machine learning modelperformance.

Story 3

Perform feature engineering by extracting seasonal crop information and preparingrelevant features for prediction.

Story 4

Split the processed dataset into training and testing datasets for model training, validation,and performance evaluation.

Epic 4: Machine Learning Model Development

Story 1

Apply the K-Means Clustering algorithm to group similar agricultural conditions and identify hidden data patterns.

Story 2

Train a Logistic Regression model using the processed agricultural dataset to predict the most suitable crops.

Story 3

Evaluate model performance using appropriate evaluation metrics such as Accuracy Precision, Recall, F1-Score, and Confusion Matrix, and compare model effectiveness.

Story 4

Save the best-performing trained model and integrate it into the application to generate crop recommendations based on user-provided soil and environmental parameters.

Epic 5: Web Application Development and Deployment

Story 1

Design and develop responsive HTML, CSS, and JavaScript pages to provide an intuitive and user-friendly interface.

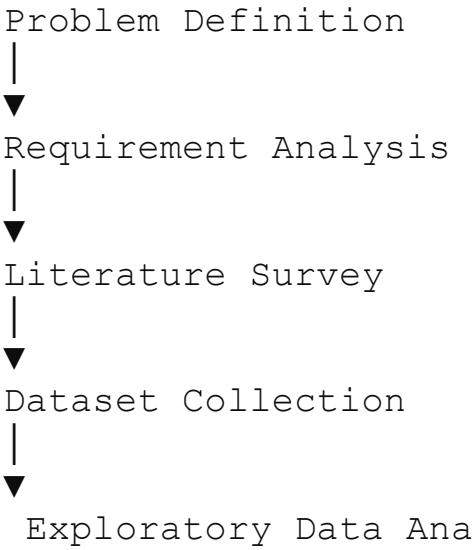
Story 2

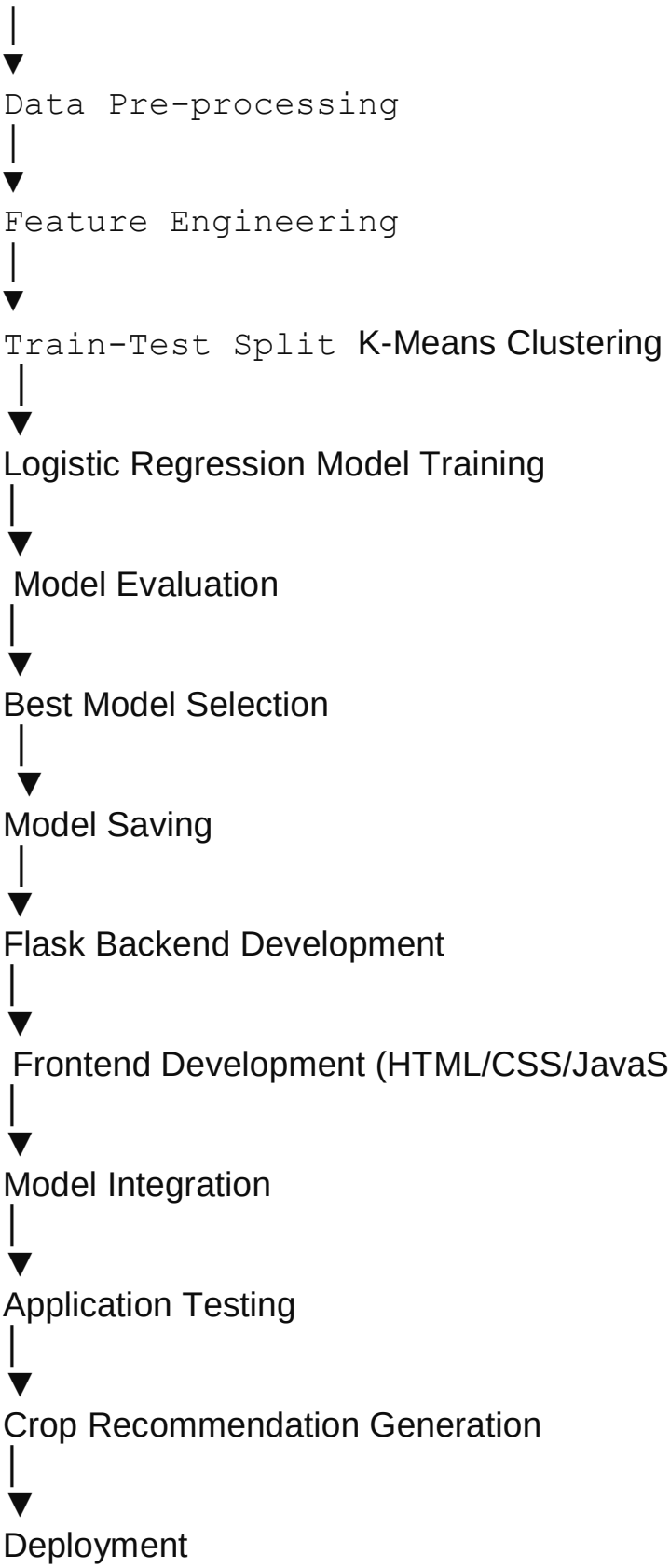
Develop the Flask backend and integrate it with the trained machine learning model, database, and prediction engine.

Story 3

Test, validate, and deploy the application to ensure accurate crop recommendations, reliable performance, and smooth user interaction.

Workflow Summary




Data Pre-processing
Feature Engineering
Train-Test Split
K-Means Clustering
Logistic Regression Model Training
Model Evaluation
Best Model Selection
Model Saving
Flask Backend Development
Frontend Development (HTML/CSS/JavaScript)
Model Integration
Application Testing
Crop Recommendation Generation
Deployment

Resources

No additional external resources are required for implementing this workflow. Theworkflow relies on open-source Python libraries, the agricultural dataset, and the Flaskweb framework to develop, test, and deploy the **OptiCrop: Smart AgriculturalProduction Optimization Engine**.

Specify the Business Problems

The **OptiCrop: Smart Agricultural Production Optimization Engine** is designed toaddress critical challenges in modern agriculture by leveraging machine learning anddata-driven decision-making. Traditional farming practices often rely on farmers'experience and intuition, which may not always account for changing environmentalconditions or soil characteristics. As a result, inappropriate crop selection can lead toreduced productivity, inefficient resource utilization, and financial losses.
This phase focuses on identifying the core business problems that the system aims tosolve and defining the objectives that guide the development of the intelligent croprecommendation platform.

Business Problems

1. Inappropriate Crop Selection

Farmers often choose crops without adequate analysis of soil nutrients and climaticconditions. This mismatch reduces crop yield, lowers product quality, and increases therisk of crop failure
.

2. Inefficient Utilization of Agricultural Resources

Resources such as fertilizers, water, land, and labor are frequently used inefficiently due tothe lack of scientific crop planning. This increases cultivation costs and negatively impactsenvironmental sustainability.

3. Changing Environmental and Climatic Conditions

Variations in temperature, rainfall, humidity, and seasonal patterns make traditional farmingdecisions less reliable. Farmers require intelligent systems capable of adaptingrecommendations to current environmental conditions.

4. Limited Access to Data-Driven Decision Support

Many farmers, particularly in rural areas, have limited access to modern technologies
thatprovide scientifically supported crop recommendations based on soil and climate data.

5. Low Agricultural Productivity

Poor crop planning and ineffective farming practices contribute to lower agricultural output,reduced profitability, and food production challenges.

6. Soil Nutrient Imbalance
Improper management of essential nutrients such as Nitrogen (N), Phosphorus (P), andPotassium (K) can degrade soil fertility and reduce long-term agricultural productivity.
7. Lack of Intelligent Prediction Systems
Traditional agricultural practices often lack predictive tools that analyze multipleenvironmental parameters simultaneously to recommend the most suitable crops.

Business Objectives

The OptiCrop system is developed to achieve the following objectives:

- Recommend the most suitable crop based on soil nutrients and environmentalconditions.
- Improve agricultural productivity through intelligent crop prediction.
- Optimize the utilization of fertilizers, water, and other farming resources.
- Support sustainable and environmentally friendly farming practices.
- Reduce financial risks associated with poor crop selection.
- Provide accurate, data-driven decision support for farmers, researchers, agribusinessorganizations, and policymakers.

Enhance overall farming efficiency using machine learning and predictive analytics.

Key Parameters Considered

The system analyzes several important agricultural parameters, including:

- Nitrogen (N)
- Phosphorus (P)
- Potassium (K)
- Soil pH
- Temperature
- Humidity
- Rainfall

Seasonal Conditions

These parameters serve as the primary inputs for the machine learning model to generateaccurate crop recommendations.

Expected Outcomes

By addressing the identified business problems, the **OptiCrop: Smart AgriculturalProduction Optimization Engine** is expected to:

- Increase crop yield and overall farm productivity.
- Improve the accuracy of crop selection.
- Reduce unnecessary use of fertilizers and water.
- Promote sustainable agricultural practices.
- Support informed decision-making through machine learning.
- Enhance profitability for farmers while minimizing environmental impact.

Resources

No additional external resources are required during this phase. The business problemanalysis is based on the project's objectives, agricultural domain knowledge, stakeholderrequirements, and the environmental and soil parameters used for intelligent croprecommendation

Business Requirements

Description

The OptiCrop: Smart Agricultural Production Optimization Engine is designed to provide an
intelligent, data-driven crop recommendation system that assists farmers, agricultural researchers,
agribusiness organizations, and policymakers in making informed cultivation decisions. The system
analyzes soil and environmental conditions using machine learning techniques to recommend the
most suitable crops, thereby improving agricultural productivity, resource utilization, and
sustainability.

Functional Business Requirements

BR-01 User Input Management: User inputs N, P, K, Temperature, Humidity, Soil pH, Rainfall,
Seasonal conditions.
BR-02 Data Validation: Validate completeness and acceptable ranges.
BR-03 Data Preprocessing: Data cleaning, normalization, feature scaling, transformation.
BR-04 Machine Learning Model Training: KNN, Logistic Regression, Decision Tree, Random

Forest, K-Means Clustering.
BR-05 Crop Recommendation: Recommend the most suitable crop.
BR-06 Prediction Results: Display real-time prediction results.
BR-07 Model Evaluation: Accuracy, Precision, Recall, F1-Score, Confusion Matrix.
BR-08 Data Visualization: Graphical visualizations for analysis and performance.
BR-09 Report Generation: Generate crop recommendation reports.
BR-10 Database Management: Securely store users, soil data, predictions, ML models, and
reports.

Non-Functional Business Requirements
Performance, Usability, Reliability, Scalability, Security, Maintainability, and Availability as
specified.

Business Goals
Improve crop selection accuracy; increase productivity; optimize water, fertilizers, and soil nutrients;
reduce crop failure; support sustainable farming; enable data-driven decisions; provide a scalable
smart agriculture platform.

Expected Business Outcomes
Accurate crop recommendations, improved resource management, higher productivity, reduced
environmental impact, increased precision agriculture adoption, and support for future IoT/cloud
integration.

Resources
No additional external resources are required. Requirements are derived from project objectives,
stakeholder needs, agricultural domain knowledge, and system capabilities.

Literature Survey
Description

This phase involves conducting a detailed study of existing agricultural recommendation systems,crop prediction technologies, and machine learning-based smart farming solutions. Various research papers, journals, case studies, online resources, and agricultural technology platforms are analyzed to understand the application of artificial intelligence and data analytics in modern agriculture.
The literature survey examines machine learning algorithms including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), Logistic Regression, Neural Networks, and clustering techniques for crop recommendation, yield prediction, soil analysis, and environmental monitoring.Their performance, strengths, and limitations are compared to identify the most suitable techniques for agricultural applications.
The study also reviews data preprocessing methods, feature engineering, dataset balancing techniques, and evaluation metrics used to improve model accuracy and reliability. Existing smart farming platforms and precision agriculture technologies are analyzed to identify research gaps, implementation challenges, and opportunities for enhancement.The findings from this survey support the selection of appropriate methodologies, system architecture, and machine learning models for the development of the **OptiCrop: Smart Agricultural Production Optimization Engine**. The survey establishes a strong foundation for efficient crop recommendation, prediction optimization, sustainable farming, and future agricultural innovations.

Resources
No additional external resources are required for this phase. The literature survey is based on published research papers, journals, conference proceedings, agricultural technology reports, and standard machine learning concepts relevant to crop recommendation systems.

Social and Business Impact
Description
The **OptiCrop: Smart Agricultural Production Optimization Engine** creates a positive impact on agriculture and society by enabling intelligent, data-driven farming decisions. Using machine learning to analyze soil nutrients and environmental conditions, the system recommends the most suitable crops, helping farmers improve productivity, reduce cultivation risks, and increase profitability.

Social Impact

- Improves farmers' decision-making through accurate crop recommendations.
- Increases crop yield and supports food security.
- Promotes sustainable farming by optimizing the use of water, fertilizers, pesticides, and soil nutrients.
- Reduces environmental degradation and minimizes resource wastage.
- Encourages the adoption of precision agriculture and smart farming technologies.
- Supports rural development by improving farmers' income and reducing financial losses caused
- by poor crop selection and changing environmental conditions.
-

Business Impact

• Assists agribusiness companies in improving agricultural planning and production strategies.
• Supports agricultural researchers with data-driven insights for crop analysis and innovation.
• Helps policymakers make informed decisions related to sustainable agriculture and food production.
• Improves supply chain planning, crop monitoring, and resource allocation.
• Enhances operational efficiency by integrating predictive analytics into agricultural decision-making.
• Provides a scalable platform for future integration with IoT devices, cloud computing, and advanced AI models.

Long-Term Benefits

The project contributes to sustainable agricultural growth, improved resource management, increased farmer profitability, technological advancement, and wider adoption of intelligent farming practices. It supports precision agriculture while strengthening food security and environmental sustainability.

Resources

No additional external resources are required. The impact assessment is derived from the objectives, expected outcomes, and practical applications of the OptiCrop system.

Download the Dataset

Description

Popular open-source agricultural datasets are available through platforms such as Kaggle and the UCI Machine Learning Repository. For this project, the **Crop_recommendation.csv** dataset is downloaded from Kaggle for agricultural prediction and crop recommendation analysis.

Dataset Link

https://www.kaggle.com/datasets/chitrakumari25/smart-agricultural-production-optimizing-engine

Dataset Download Steps

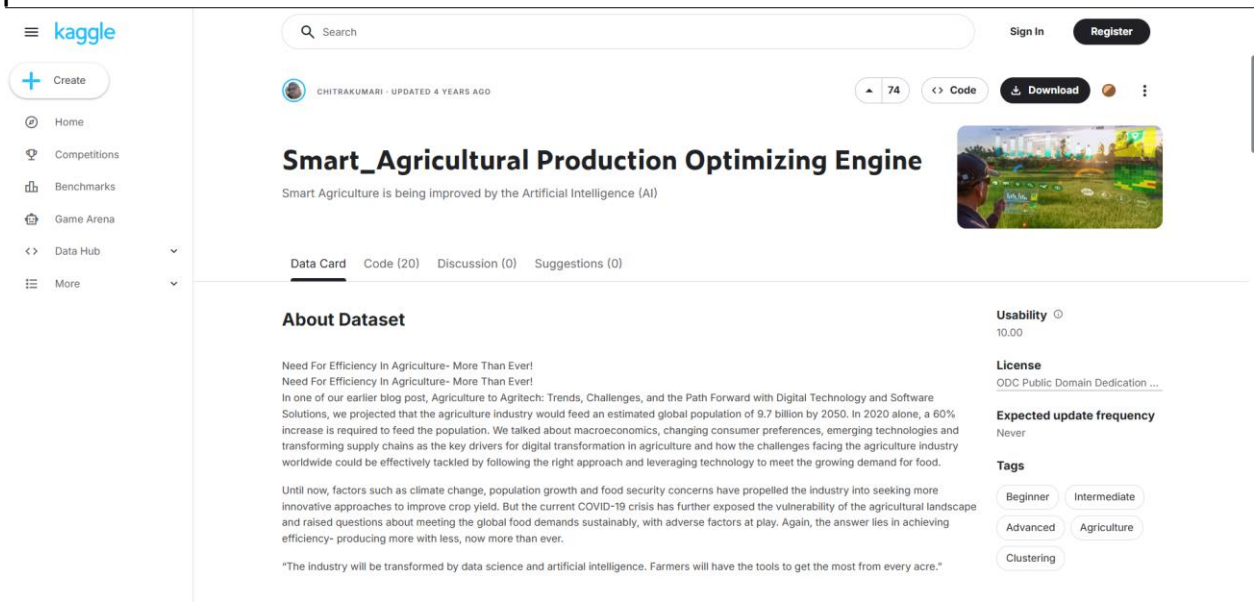
1. Visit the Kaggle dataset page.
2. Sign in or register for a Kaggle account.
3. Click the **Download** button.
4. Extract the ZIP file.
5. Copy **Crop_recommendation.csv** into the project folder.

Purpose

The dataset is used for exploratory data analysis, preprocessing, machine learning model training,
evaluation, and intelligent crop recommendation.

Note

Additional visualization and analytical techniques may be applied to improve prediction
performance.



Resources
No resources available.

Importing the Libraries

Description
The required Python libraries are imported to perform agricultural data analysis, machine learning operations, and visualization within the OptiCrop system. Libraries such as NumPy and Pandas are used for numerical computation and dataset handling, while Matplotlib and Seaborn are used for graphical visualization and analytical plotting. Scikit-learn modules are integrated for machine learning model development,clustering, dataset splitting, and performance evaluation. The visualization style used in this project is **FiveThirtyEight(fivethirtyeight)**, which helps generate clean and professional analytical graphs for agricultural data interpretation and crop prediction analysis.

```
import pandas as pd
import numpy as np
pd.set_option('max_colwidth', 20)
pd.set_option('display.max_columns',None)
pd.set_option('display.max_rows',50)
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
plt.rcParams['figure.figsize'] = (12,8)
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
from ipywidgets import interact
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```


```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix,
classification_report
```

Resources
No resources available.

Read the Dataset – Epic 2: Data Collection and Analysis

Description
The agricultural dataset is loaded into the OptiCrop system using the Pandas **read_csv()** function for further preprocessing and analysis. The dataset contains important agricultural parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, rainfall, and crop labels required for intelligent crop recommendation workflows. After loading the dataset, the initial records are displayed using the **head()** function to understand the dataset structure, feature columns, and agricultural data distribution. Proper dataset reading and validation ensure smooth machine learning analysis and accurate crop prediction processing.

```
import pandas as pd
data =
pd.read_csv('/content/gdrive/MyDrive/SmartBridge/Projec
ts/OptiCrop/Crop_recommendation.csv')
data.head()
```

```
data=pd.read_csv('/content/gdrive/MyDrive/SmartBridge/Projects/OptiCrop/Crop_recommendation.csv')
data.head()
```

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

Resources
No resources available.

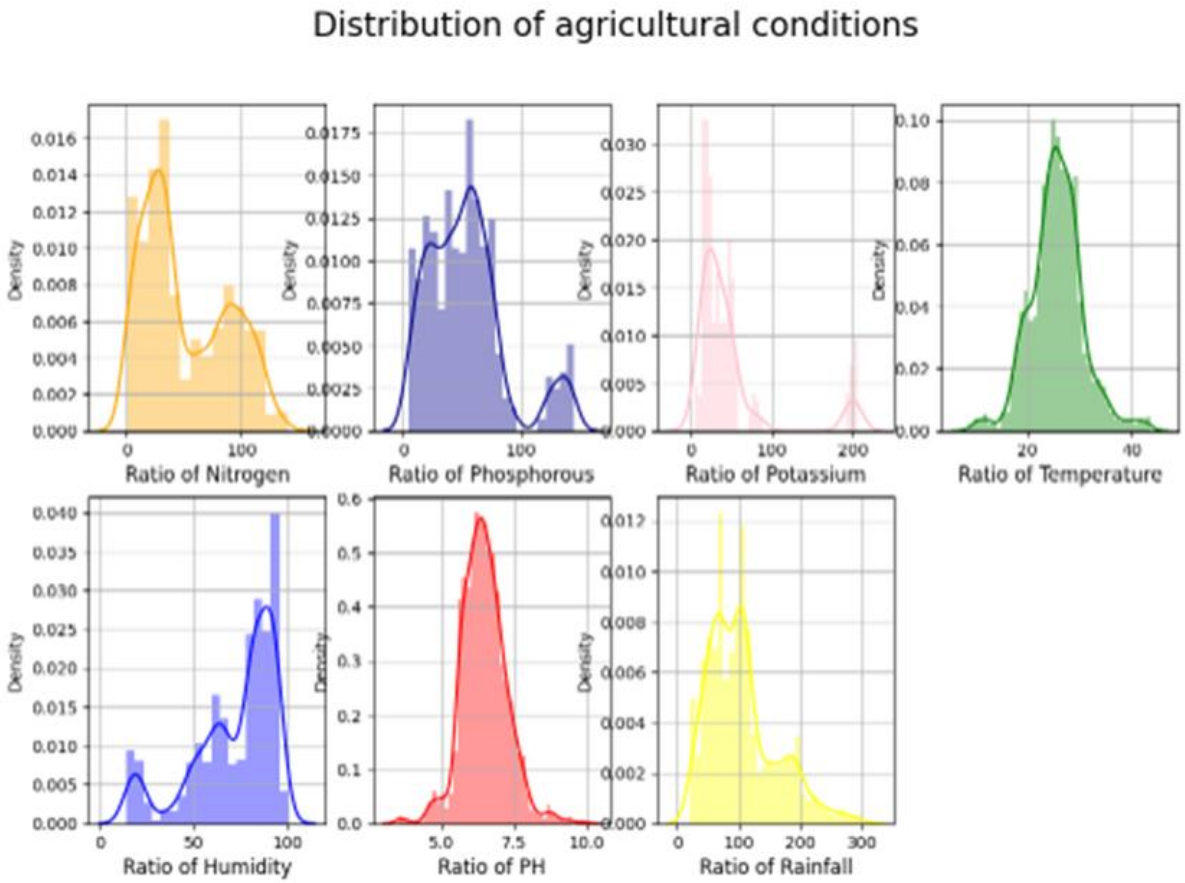
Univariate Analysis – Epic 2: Data Collection and Analysis

Description

Univariate analysis is performed to study individual agricultural features within the crop recommendation dataset. Visualization techniques such as histogram plots with density estimation (distplot/histplot with KDE) and count plots are used to analyze the distribution of environmental parameters including Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall. This analysis helps identify data patterns, feature distributions, skewness, and variability, providing valuable insights for intelligent crop prediction and smart farming optimization.

Purpose

- Understand the distribution of each feature.
- Detect skewness and unusual patterns.
- Support feature engineering and preprocessing.
- Improve machine learning model performance.



```
# Example
import seaborn as sns
import matplotlib.pyplot as plt
data.hist(figsize=(12,8))
plt.show()
```

Resources

No resources available.

Bivariate Analysis – Epic 2: Data Collection and Analysis

Description

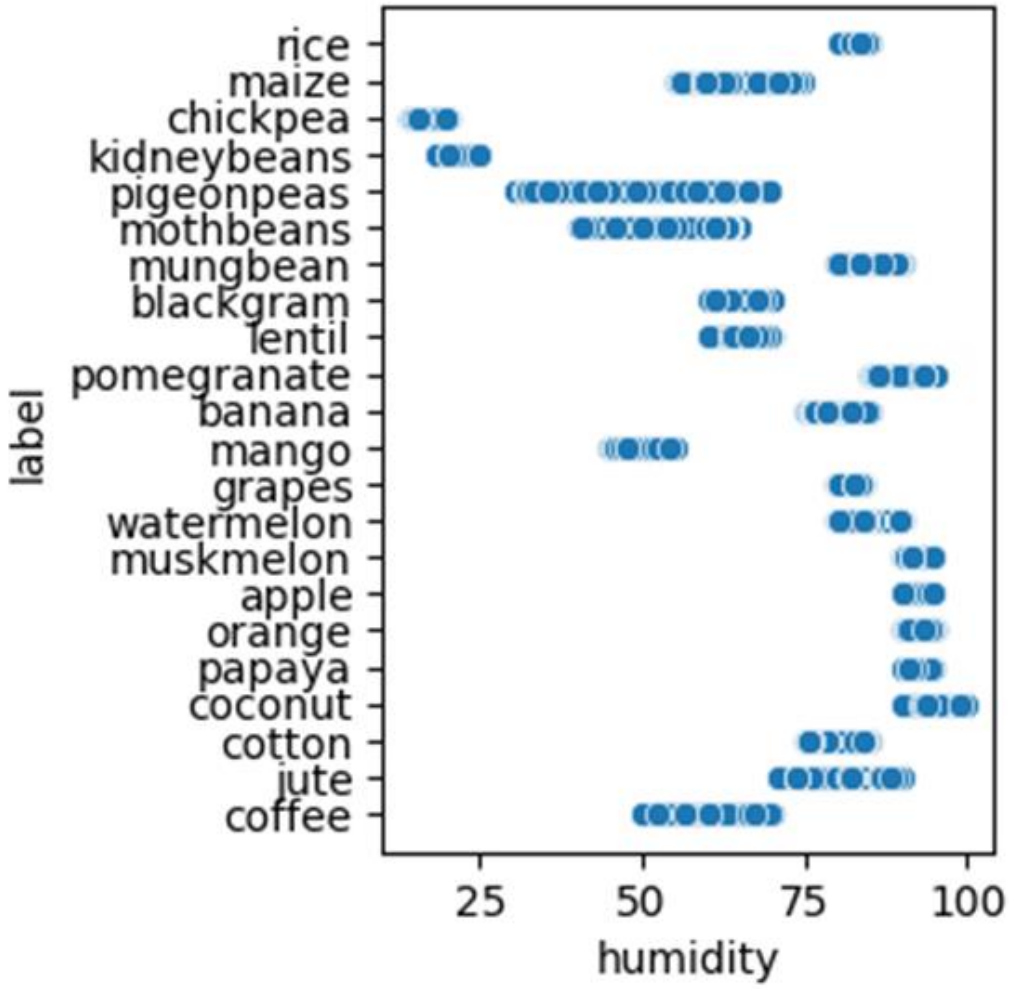
Bivariate analysis is used to identify relationships between two agricultural features within the crop recommendation dataset. In this analysis, the relationship between **humidity** and **crop labels** is visualized using a scatter plot to understand how humidity influences crop suitability. This analysis helps identify environmental patterns that support intelligent crop prediction and agricultural decision-making.

Objectives

• Examine the relationship between humidity and crop type.
• Identify trends and variations among crops.
• Support feature selection and machine learning model development.
• Improve crop recommendation accuracy through data exploration.

```
plt.subplot(2,4,7)
sns.scatterplot(x=df['humidity'],y=df['label'])
```

```
<Axes: ><Axes: xlabel='humidity', ylabel='label'>
```




```
plt.subplot(2,4,7)
sns.scatterplot(x=df['humidity'], y=df['label'])
plt.show()
```

Resources

No resources available.

Multivariate Analysis – Epic 2: Data Collection and Analysis

Description

Multivariate analysis is used to study relationships among multiple agricultural features simultaneously within the crop recommendation dataset. Visualization techniques such as count plots and descriptive statistical analysis are used to examine Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall. This analysis helps identify agricultural patterns, understand feature distributions, and supports intelligent crop prediction and smart farming optimization workflows.

Descriptive Analysis

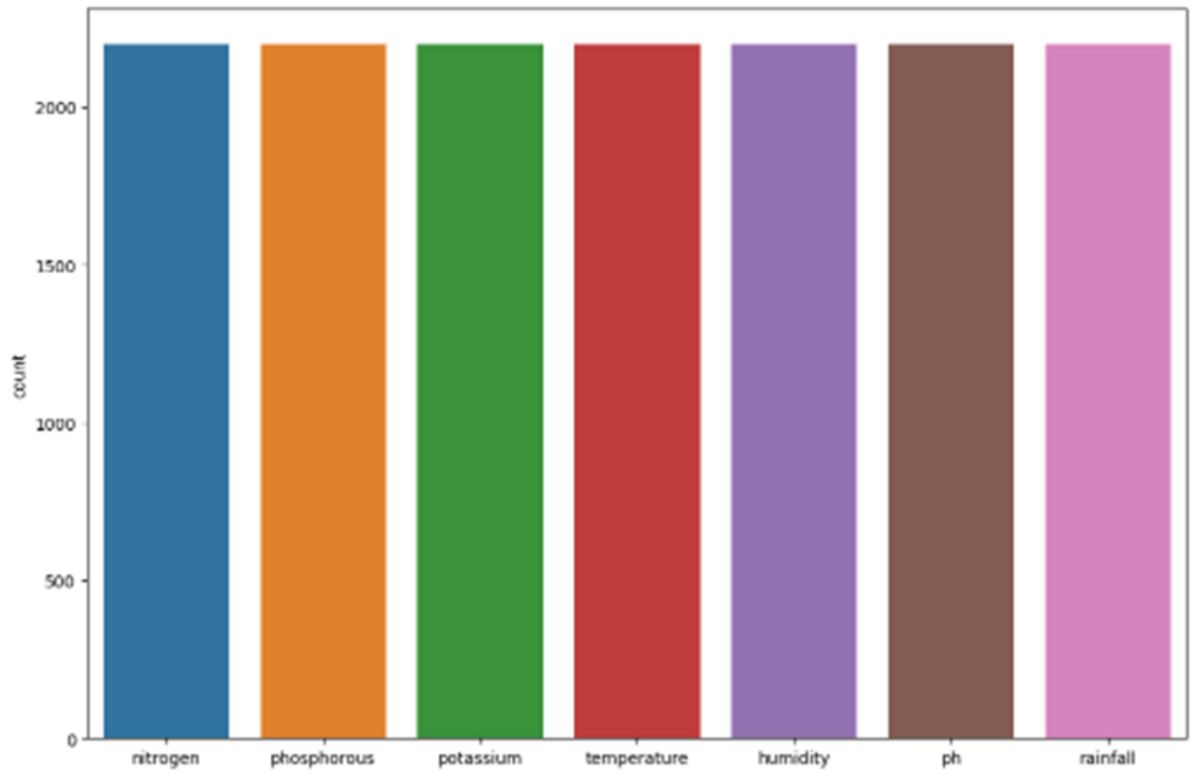
Descriptive statistics summarize the dataset by providing the count, mean, standard deviation,minimum, maximum, and quartile values for each agricultural parameter. These measures help understand the central tendency, variability, and overall quality of the dataset before machine learning model development.

Objectives

- Analyze relationships among multiple variables.
- Understand feature distributions and statistics.
- Support preprocessing and feature engineering.
- Improve crop recommendation model performance

```
sns.countplot(df)
```

```
<Axes: ylabel='count'>
```




```
df.describe()
```

	nitrogen	phosphorous	potassium	temperature	humidity	ph	rainfall
count	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000
mean	50.551818	53.362727	48.149091	25.616244	71.481779	6.469480	103.463655
std	36.917334	32.985883	50.647931	5.063749	22.263812	0.773938	54.958389
min	0.000000	5.000000	5.000000	8.825675	14.258040	3.504752	20.211267
25%	21.000000	28.000000	20.000000	22.769375	60.261953	5.971693	64.551686
50%	37.000000	51.000000	32.000000	25.598693	80.473146	6.425045	94.867624
75%	84.250000	68.000000	49.000000	28.561654	89.948771	6.923643	124.267508
max	140.000000	145.000000	205.000000	43.675493	99.981876	9.935091	298.560117

Resources

No resources available.

Checking for Null Values – Epic 3: Data Pre-Processing

Description

The dataset is examined for missing and null values before preprocessing and machine learning operations. Functions such as **df.shape**, **df.info()**, and **df.isnull().sum()** are used to analyze the dataset structure, data types, total records, and missing values. This validation step ensures that the agricultural dataset is complete, consistent, and ready for preprocessing and machine learning model development.

Purpose

- Verify the dataset dimensions.
- Check data types and non-null values.
- Identify missing or null records.
- Ensure clean and reliable data for crop prediction.

Observation

The dataset contains 2,200 records and 8 columns. All columns have complete (non-null) values,indicating that no missing data is present

```
[5] data.shape
```

```
(2200, 8)
```

```
[4] data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2200 entries, 0 to 2199
Data columns (total 8 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   N            2200 non-null   int64  
 1   P            2200 non-null   int64  
 2   K            2200 non-null   int64  
 3   temperature  2200 non-null   float64
 4   humidity     2200 non-null   float64
 5   ph           2200 non-null   float64
 6   rainfall     2200 non-null   float64
 7   label        2200 non-null   object 
dtypes: float64(4), int64(3), object(1)
memory usage: 137.6+ KB
```

```
[8]  df.isnull().sum()

     nitrogen       0
     phosphorous    0
     potassium      0
     temperature    0
     humidity       0
     ph             0
     rainfall       0
     label          0
     dtype: int64
```

Resources
No resources available.

Handling Outliers – Epic 3: Data Pre-Processing

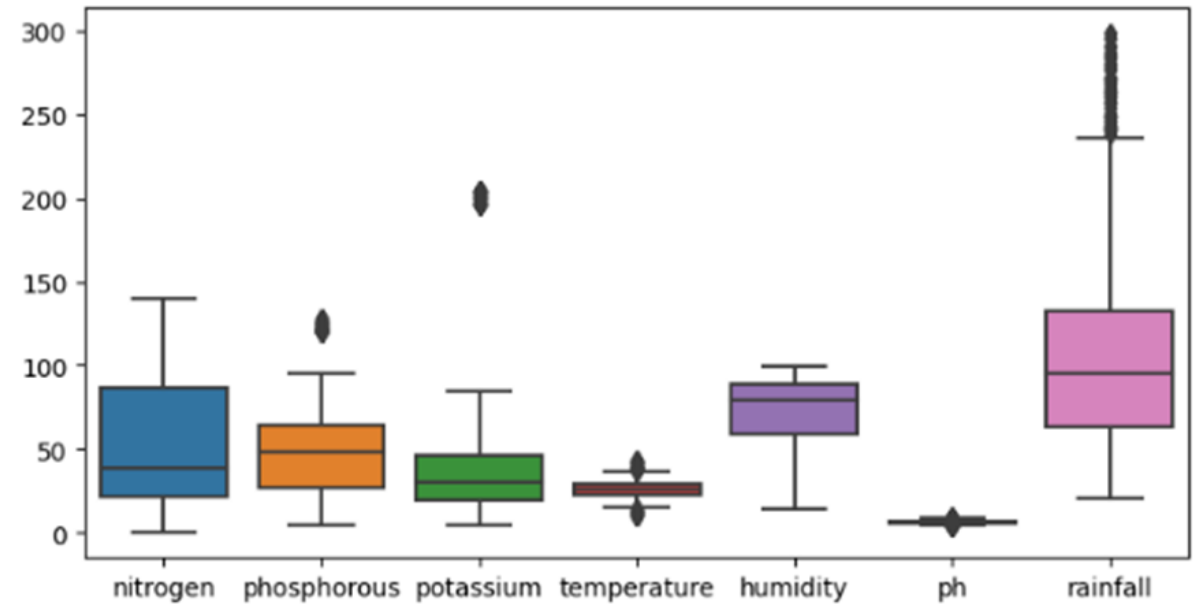
Description

Outliers are extreme data values that differ significantly from the majority of observations and can negatively affect the performance of machine learning models. In the **OptiCrop: Smart Agricultural Production Optimization Engine**, outliers are identified visually using **boxplots** and mathematically using the **Interquartile Range (IQR)** method.

The boxplot provides a graphical representation of the distribution of agricultural parameters such as **Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall**, making it easier to detect unusually high or low values. From the visualization, the **Potassium** feature contains noticeable outliers, while **Rainfall** also shows several extreme values.

```
plt.figure(figsize=(8,4))
sns.boxplot(df)
```

```
<Figure size 800x400 with 0 Axes><Axes: >
```




```
Q1 = df['phosphorous'].quantile(0.25)
Q3 = df['phosphorous'].quantile(0.75)
IQR = Q3 - Q1    #IQR is interquartile range.

filter = (df['phosphorous'] >= Q1 - 1.5 * IQR) & (df['phosphorous'] <= Q3 + 1.5 *IQR)
df=df.loc[filter]
```

The **Interquartile Range (IQR)** is calculated as:

IQR = Q3 − Q1

where:

- **Q1** = First Quartile (25th percentile)
- **Q3** = Third Quartile (75th percentile)

The acceptable range of values is determined using the following formulas:

- **Lower Bound = Q1 − 1.5 × IQR**
- **Upper Bound = Q3 + 1.5 × IQR**

Any values falling outside these limits are treated as outliers.

After identifying the outliers, suitable preprocessing techniques such as **IQR filtering**, **winsorization**, or **logarithmic transformation** can be applied to reduce their influence while preserving useful agricultural information. Handling outliers improves data quality, enhances model stability, and increases the accuracy of crop recommendation predictions.

Purpose

- Detect abnormal agricultural observations.
- Reduce the impact of extreme values.
- Improve data quality before model training.
- Increase machine learning prediction accuracy.
- Produce a more balanced feature distribution.

Outlier Detection Method

Interquartile Range (IQR)

```
IQR = Q3 − Q1

Lower Bound = Q1 − 1.5 × IQR

Upper Bound = Q3 + 1.5 × IQR
```

Observation

- The boxplot indicates that the **Potassium (K)** attribute contains significant outliers.
- The **Rainfall** feature also contains several extreme observations.
- These outliers can be handled using **IQR filtering** or **log transformation** before training the machine learning models.

Expected Outcome

After handling outliers, the dataset becomes cleaner and more consistent, resulting in improved machine learning performance, better crop recommendation accuracy, and more reliable agricultural predictions.

Resources

No resources available.

Extracting Seasonal Crops - Epic 3: Data Pre-Processing

Seasonal crop extraction groups crops according to environmental conditions such as temperature,humidity, and rainfall. This preprocessing step helps identify crops suitable for summer, winter, and rainy seasons, improving recommendation accuracy in the OptiCrop system.

Objective

To classify crop labels into seasonal groups using threshold-based filtering on climatic features.

Methodology

• Summer: Temperature > 30°C and Humidity > 50%
• Winter: Temperature < 20°C and Humidity > 30%
• Rainy: Rainfall > 200 mm and Humidity > 50%
The dataset is filtered using these conditions, and unique crop labels are extracted for each season.

Season-wise Results

Summer Crops: pigeonpeas, mothbeans, blackgram, mango, grapes, orange, papaya
Winter Crops: maize, pigeonpeas, lentil, pomegranate, grapes, orange
Rainy Crops: rice, papaya, coconut

```
print("Summer crops")
print(df[(df['temperature']>30) & (df['humidity']>50)]['label'].unique())
print("-----------------------------------------------")
print("Winter crops")
print(df[(df['temperature']<20) & (df['humidity']>30)]['label'].unique())
print("-----------------------------------------------")
print("Rainy crops")
print(df[(df['rainfall']>200) & (df['humidity']>50)]['label'].unique())
print("-----------------------------------------------")
```

```
Summer crops
['pigeonpeas' 'mothbeans' 'blackgram' 'mango' 'grapes' 'orange' 'papaya']
-----------------------------------------------
Winter crops
['maize' 'pigeonpeas' 'lentil' 'pomegranate' 'grapes' 'orange']
-----------------------------------------------
Rainy crops
['rice' 'papaya' 'coconut']
```

Conclusion

Extracting seasonal crops organizes agricultural data into climate-specific categories, enabling accurate crop recommendation, reducing preprocessing complexity, and improving the prediction performance of the OptiCrop Smart Agricultural Production Optimization Engine

Splitting Data into Train and Test Sets - Epic 3: Data Pre-Processing

The dataset is separated into feature variables (X) and the target variable (y). The Scikit-learn train_test_split() function divides the data into training and testing sets. This enables the machine learning model to learn from the training data and evaluate its performance on unseen testing data.

Methodology

1. Separate input features (X) and target label (y).
2. Use train_test_split(X, y, test_size=0.2, random_state=0).
3. Allocate 80% of the data for training and 20% for testing.
4. Preserve reproducibility using a fixed random state.

Dataset Shapes

Original Features (X): (2062, 7)

Target Labels (y): (2062,)
Training Features: (1649, 7)
Testing Features: (413, 7)

```
y=df['label']
x=df.drop(['label'],axis=1)

print("Shape of x",x.shape)
print("Shape of x=y",y.shape)
```

```
Shape of x (2062, 7)
Shape of x=y (2062,)
```

```
from sklearn.model_selection import train_test_split

x_train,x_test,y_train,y_test =train_test_split(x,y,test_size=0.2,random_state=0)

print("The shape of x train",x_train.shape)
print("The shape of x test",x_test.shape)
print("The shape of y train",x_train.shape)
print("The shape of y test",x_test.shape)
```

```
The shape of x train (1649, 7)
The shape of x test (413, 7)
The shape of y train (1649, 7)
The shape of y test (413, 7)
```

Conclusion

Splitting the dataset into training and testing sets ensures unbiased model evaluation, prevents overfitting, and improves the reliability and generalization capability of the OptiCrop Smart Agricultural Production Optimization Engine.

K-Means Clustering - Epic 4: Model Building

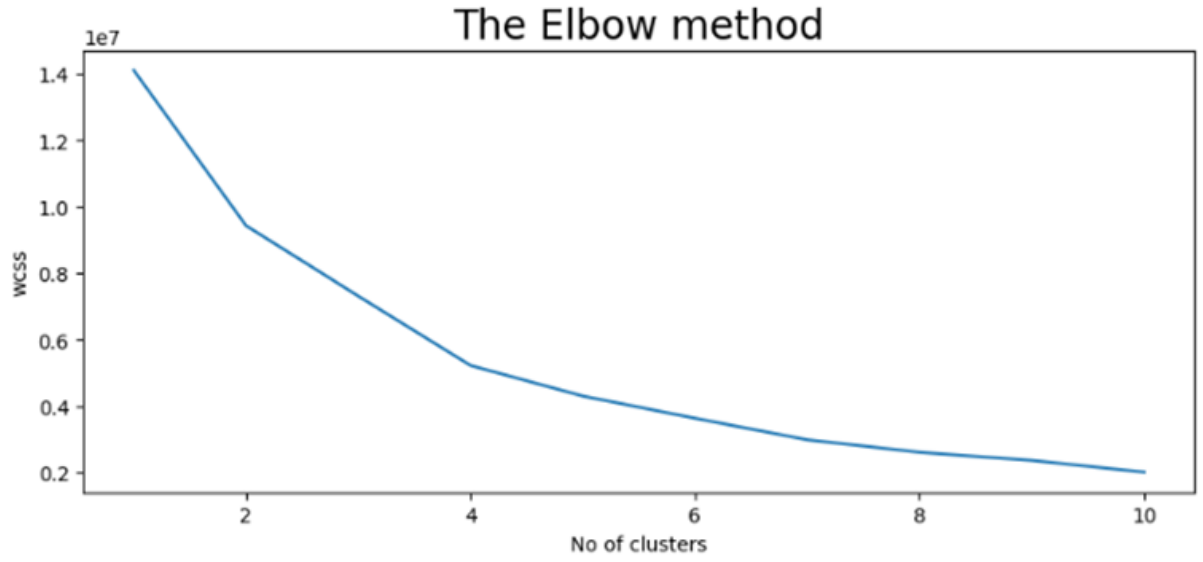
K-Means Clustering is an unsupervised machine learning algorithm used to group agricultural data with similar environmental characteristics. It helps identify hidden crop patterns and supports intelligent crop analysis.

Elbow Method

The Elbow Method determines the optimal number of clusters by plotting the Within-Cluster Sum of Squares (WCSS) against different cluster values. The point where the decrease in WCSS begins to level off is selected as the optimal number of clusters. In this project, four clusters were selected.

```
#el-bow method used to find out no of clusters and determine the optimum number of clusters within the dataset.
plt.rcParams['figure.figsize']=(10,4)
wcss=[]
for i in range(1,11):
    km=KMeans(n_clusters=i,init="k-means++",max_iter=300,n_init=10,random_state=0)
    km.fit(x)
    wcss.append(km.inertia_)

plt.plot(range(1,11),wcss)
plt.title("The Elbow method",fontsize=20)
plt.xlabel("No of clusters")
plt.ylabel("wcss")
plt.show()
```



Training the Model

The K-Means model is trained using KMeans(n_clusters=4, init='k-means++', max_iter=300,n_init=10, random_state=0). The fit_predict() method assigns each crop record to a cluster based on similarity among environmental features

Logistic Regression - Epic 4: Model Building

Logistic Regression is a supervised machine learning algorithm used to predict the most suitable crop based on agricultural and environmental features such as nitrogen, phosphorus, potassium, temperature, humidity, pH, and rainfall. The model learns from labeled training data and generates crop predictions for unseen data.

Model Training

The Logistic Regression model is created using the Scikit-learn library. The **fit()** method trains the model using the training dataset, while the **predict()**

method generates predictions for the testing dataset. During training, a convergence warning may appear if the maximum number of iterations is reached; this can be addressed by increasing **max_iter** or scaling the input features.

```
# LOGUSTIC REGRESSION
from sklearn.linear_model import LogisticRegression

model=LogisticRegression()
model.fit(x_train,y_train)
y_pred=model.predict(x_test)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:458: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

```
LogisticRegression()
```

Workflow

1. Import LogisticRegression from Scikit-learn.
2. Create the Logistic Regression model.
3. Train the model using **fit(x_train, y_train)**.
4. Predict crop labels using **predict(x_test)**.
5. Evaluate the model using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

Conclusion

The Logistic Regression model provides an efficient baseline classifier for crop recommendation. By learning relationships between environmental conditions and crop labels, it supports accurate crop prediction and contributes to the intelligent decision-making capabilities of the OptiCrop system.

Evaluating Model Performance and Saving the Best Model **Epic 4: Model Building**

After training the machine learning models, their performance is evaluated using classification metrics including Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and ROC-AUC Score.These metrics measure prediction quality and help identify the model that generalizes best to unseen data.

Performance Evaluation
The classification report summarizes the performance for each crop class. The Logistic Regression model achieved an overall accuracy of approximately **94%**, demonstrating reliable prediction capability across the crop categories. Macro and weighted averages indicate balanced performance.

Predict the Best Crop Based on Given Parameters
Epic 4: Model Building

After training and saving the machine learning model, it is used to predict the most suitable crop based on environmental and soil parameters provided by the user. The trained Logistic Regression model receives feature values such as nitrogen, phosphorus, potassium, temperature, humidity,pH, and rainfall, and returns the recommended crop.

Prediction Process
1. Load the trained model (model.pkl).
2. Collect user input values.
3. Convert the inputs into a NumPy array.
4. Call **model.predict()** to generate the crop recommendation.
5. Display the predicted crop to the user through the application interface.

```
prediction=model.predict((np.array([[105,35,40,25,64,7,160]])))

print("The suggested crop for given climatic condition is: ",prediction)
```

```
The suggested crop for given climatic condition is:  ['coffee']
/usr/local/lib/python3.10/dist-packages/sklearn/base.py:439: UserWarning: X does not have valid feature names, but LogisticRegression was fitted with feature names
  warnings.warn(
```

Prediction Example
Input Parameters:
Nitrogen = 105, Phosphorus = 35, Potassium = 40, Temperature = 25°C, Humidity = 64%, pH = 7,
Rainfall = 160 mm.
Predicted Crop: Coffee

Conclusion
The prediction module enables intelligent crop recommendation by analyzing user-provided agricultural parameters. This functionality helps farmers make informed decisions, improves crop productivity, and

demonstrates the practical deployment of the trained OptiCrop machine learning model.

Building HTML Pages - Epic 5: Application Building

This phase focuses on designing the web interface of the OptiCrop application using HTML. The
application provides intuitive navigation, project information, and a crop prediction form that
interacts with the Flask backend to generate intelligent crop recommendations.

Home Page

The Home page contains a navigation bar with links to Home, About, and FindYourCrop pages. It introduces the purpose of the OptiCrop system and enables smooth navigation.

```
<body>

<div class="topnav">
  <a class="active" href="#home">Home</a>
  <a href="{{ url_for('about') }}">About</a>
  <a href="{{ url_for('findyourcrop') }}">FindYourCrop</a>
</div>

<div style="padding-left:16px">
  <h2>Opti Crop Agricultural Optimisation</h2>
  <p>The main purpose of the OptiCrop aims to develop an advanced software system
</div>

</body>
</html>
```

About Page

The About page explains the objectives of the OptiCrop project, highlighting the use of machine learning and environmental parameters to improve agricultural productivity and sustainable farming.

Build Python Backend Code - Epic 5: Application Building

The Python backend is developed using the Flask framework to connect the web interface with the trained machine learning model. The backend manages routing, loads the saved model, processes user inputs, generates crop predictions, and runs the web application.

Backend Initialization

This section imports the required libraries (NumPy, Flask, and Pickle), creates the Flask application object, and loads the trained model (model.pkl).

```
import numpy as np
from flask import Flask, request, render_template
import pickle
```

URL Routing

Flask route decorators define the Home, About, and FindYourCrop pages and render their HTMLtemplates.

```
@app.route('/')

def home():
    return render_template('home.html')

@app.route('/about')
def about():
    return render_template('about.html')

@app.route('/findyourcrop')
def findyourcrop():
    return render_template('findyourcrop.html')
```

Prediction Function

The /predict route accepts POST requests, converts form values to numeric features, invokes model.predict(), and returns the predicted crop to the HTML page.

```
@app.route('/predict',methods=['POST'])
def predict():
    int_features= [float(x) for x in request.form.values()]
    features=[np.array(int_features)]
    prediction=model.predict(features)

    output = prediction[0]
    return render_template('findyourcrop.html',prediction_text='Best crop for given conditions is {}'.format(output))
```

Main Function

The application is started using app.run(debug=True), allowing local execution and testing in a webbrowser.

```
if __name__=="__main__":
    app.run(debug=True)
```

Run the Application - Epic 5: Application Building

This phase demonstrates how to execute the OptiCrop Flask application and verify that the integrated machine learning model works correctly through the web interface.

Execution Steps

1. Open the project in Visual Studio Code.
2. Import all project folders.
3. Run **app.py**.
4. Click the server URL shown in the terminal.
5. Access the application in a web browser and navigate through the pages.

Home Page

The Home page is displayed after launching the application and provides navigation to the About and FindYourCrop pages.

About Page

The About page describes the objectives of OptiCrop and explains how machine learning supports intelligent crop recommendation

Conclusion

The OptiCrop: Smart Agricultural Production Optimization Engine project successfully demonstrates the application of machine learning and web technologies for intelligent crop recommendation. The project began with comprehensive data preprocessing, including handling missing values, outlier detection, seasonal crop extraction, and train-test data splitting. These preprocessing steps improved data quality and prepared the dataset for effective model training. Machine learning models such as K-Means Clustering and Logistic Regression were developed to analyze agricultural patterns and predict suitable crops based on soil nutrients and environmental conditions. Model evaluation using Accuracy, Precision, Recall, and F1-Score identified the best performing model, which was saved for deployment using Pickle. A Flask-based web application was then designed to integrate the trained model with an interactive HTML interface. Users can enter Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall values to receive real-time crop recommendations. This provides a practical decision-support tool for farmers and agricultural stakeholders.

Project Outcomes

• Improved agricultural decision-making through intelligent crop prediction.

• Successful implementation of machine learning algorithms for crop recommendation.
• Development of an interactive Flask web application.
• Deployment-ready prediction model using model.pkl.
• User-friendly interface for real-time crop recommendation.

Future Scope

The OptiCrop system can be further enhanced by integrating real-time weather data, IoT-based soil sensors, satellite imagery, fertilizer recommendation, pest and disease prediction, multilingual support, and cloud deployment. Future versions may also incorporate advanced deep learning models to improve prediction accuracy and scalability for precision agriculture.

Final Remark

Overall, OptiCrop provides an efficient, reliable, and intelligent crop recommendation platform that combines machine learning with modern web technologies. The project demonstrates how data-driven solutions can improve agricultural productivity, optimize resource utilization, and support sustainable farming practices.